

1. Objetivo

O objetivo deste projeto é desenvolver um simulador de escalonamento de tarefas críticas utilizando os algoritmos Rate Monotonic(Rate) e Earliest Deadline First(EDF). O programa deve realizar a execução preemptiva das tarefas, respeitando seus períodos, deadlines e tempos de execução. Também deve identificar tarefas que concluíram sua execução, perderam deadlines ou permaneceram ativas ao final da simulação. Por fim, busca-se comparar o comportamento dos dois algoritmos em diferentes conjuntos de tarefas.

2. Checklist dos requisitos

Requisito do enunciado	Status	Como testar / evidência
Leitura do tempo total e das tarefas a partir do arquivo de entrada	OK	Executar o programa com um arquivo de entrada válido e verificar a simulação gerada
Implementação do algoritmo Rate Monotonic	ok	Executar ./scheduler rate entrada.txt e verificar o arquivo rate_lfass.out
Implementação do algoritmo Earliest Deadline First (EDF)	ok	executar ./scheduler edf entrada.txt e verificar o arquivo edf_lfass.out
Execução preemptiva das tarefas	ok	Utilizar um conjunto de tarefas em que uma tarefa de maior prioridade interrompa outra
Prioridade do Rate Monotonic pelo menor período	ok	Testar tarefas com períodos diferentes e verificar a ordem

		de execução
Prioridade do EDF pelo menor deadline absoluto	ok	Testar tarefas com deadlines absolutos diferentes e verificar a ordem de execução
Desempate pela ordem das tarefas no arquivo	ok	Testar duas tarefas com mesma prioridade e verificar qual é executada primeiro
Identificação de tarefas que perdem o deadline	ok	Utilizar uma entrada em que uma tarefa não consiga terminar antes do deadline
Descarte do restante da instância após perda do deadline	ok	Verificar a seção LOST DEADLINES e a execução da próxima instância
Identificação das execuções completas	ok	Verificar a seção COMPLETE EXECUTION no arquivo de saída
Identificação das tarefas ainda ativas ao final da simulação (KILLED)	ok	Utilizar uma simulação que termine enquanto uma tarefa ainda possui tempo restante
Registro dos períodos de ociosidade	ok	Utilizar uma entrada em que não haja tarefa pronta para executar e verificar idle for ... units
Geração do arquivo de saída correspondente ao algoritmo	ok	Verificar a criação de rate_lfass.out ou edf_lfass.out
Validação dos argumentos de execução	ok	Executar com quantidade incorreta de argumentos ou algoritmo inválido
Validação dos dados das tarefas	ok	Testar valores não positivos, $C > D$ e $D > P$
Alocação dinâmica e liberação da memória das tarefas	ok	

		Verificar a implementação no scheduler.c
Análise comparativa entre Rate Monotonic e EDF	ok	estar um conjunto de tarefas em que RATE perca deadline e EDF não, explicando a diferença entre os critérios de prioridade

3. Como Reproduzir

Compilar

Para compilar o projeto, execute os seguintes comandos:

```
make clean
make
```

Executar

O programa deve ser executado informando o algoritmo de escalonamento e o arquivo de entrada:

```
./scheduler rate input.txt
```

Para executar utilizando o algoritmo EDF:

```
./scheduler edf input.txt
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. src/scheduler.c → responsável pela implementação do simulador, incluindo leitura e validação das tarefas, execução dos algoritmos Rate Monotonic e EDF, controle das instâncias, deadlines, preempções, períodos de ociosidade e geração dos resultados.

2. Makefile → responsável pela compilação do programa e pela limpeza dos arquivos compilados.
3. evidencias.log → responsável pelo registro das evidências e testes realizados durante o desenvolvimento.
4. rate_ifass.out → armazena os resultados da simulação realizada utilizando o algoritmo Rate Monotonic, incluindo a sequência de execução, períodos de ociosidade, deadlines perdidos, execuções completas e tarefas mortas.
5. edf_ifass.out → armazena os resultados da simulação realizada utilizando o algoritmo Earliest Deadline First(EDF), apresentando as mesmas informações da execução do Rate Monotonic.

Decisões técnicas:

1. Utilização de uma lista encadeada para armazenar as tarefas → permite armazenar dinamicamente a quantidade de tarefas presente no arquivo de entrada → facilita o gerenciamento das tarefas durante a simulação.
2. Utilização da própria estrutura task para representar as tarefas → evita a criação de uma estrutura adicional apenas para os nós da lista → mantém as informações da tarefa e o encadeamento em uma única estrutura.
3. Implementação dos algoritmos Rate Monotonic e EDF no mesmo simulador → permite utilizar a mesma estrutura de tarefas e comparar diretamente os dois critérios de prioridade → facilita a análise dos resultados dos diferentes algoritmos.
4. Geração de arquivos de saída separados para cada algoritmo → permite visualizar e comparar os resultados do Rate Monotonic e do EDF sem misturar as informações → facilita a análise comparativa entre os dois escalonadores.
5. Verificação das tarefas ativas ao final da simulação → foi necessário verificar se ainda existiam tarefas com execução pendente quando o tempo total da simulação chegasse ao fim → permitiu contabilizar corretamente essas instâncias como KILLED e apresentar essa informação na saída do programa.

5. Estratégias e Diário de Desenvolvimento

O desenvolvimento do projeto foi feito por partes, começando pela estrutura básica do programa e depois adicionando as funcionalidades do escalonador. Eu preferi implementar uma parte, testar se estava funcionando corretamente e só depois continuar para a próxima.

Primeiro implementei a estrutura da tarefa e a leitura do arquivo de entrada. Nessa parte foram adicionadas as informações de nome, período, deadline e burst, além das validações dos valores recebidos. Também foram feitos testes com arquivos válidos e inválidos para verificar se o programa identificava corretamente os erros de entrada.

Depois implementei a lista encadeada para armazenar as tarefas. A escolha foi utilizar a task para guardar as informações da tarefa e também o ponteiro para a próxima tarefa(lista encadeada). Depois de implementar essa parte, testei se todas as tarefas estavam sendo lidas e armazenadas corretamente.

Em seguida comecei a implementação da simulação dos escalonadores. Primeiro foi implementada a lógica do Rate Monotonic, utilizando o menor período como critério de prioridade. Depois foi adicionada a lógica do EDF, utilizando o menor deadline absoluto. Também foram realizados testes com tarefas que precisavam ser interrompidas para verificar se a preempção estava funcionando corretamente.

Durante o desenvolvimento, percebi que o programa não estava verificando as tarefas que ainda estavam

ativas quando o tempo total da simulação terminava. Foi então adicionada uma verificação após o término do while, contando como KILLED as tarefas que ainda estavam ativas e possuíam tempo de execução restante. Depois disso, o caso foi testado novamente e o resultado passou a apresentar corretamente a quantidade de tarefas mortas.

Depois foram adicionados os períodos de ociosidade da CPU. a simulação passou a identificar os momentos em que não havia nenhuma tarefa pronta para executar e registrar esses períodos no arquivo de saída. Também foram feitos testes em que a CPU ficava ociosa entre duas execuções e no final da simulação.

Por fim, foram realizados testes específicos para verificar a perda de deadlines, o tratamento de novas instâncias das tarefas, os empates entre tarefas e os casos em que uma tarefa termina exatamente no instante do deadline. Também foram testadas as entradas inválidas, como valores negativos, zero, campos faltando, valores não numéricos, $c > d$ e $d > p$.

Durante os testes do EDF também foi identificado um erro na lógica de escolha da tarefa. O critério de comparação foi alterado propositalmente para verificar se o teste conseguia detectar uma escolha incorreta. O resultado mostrou que a tarefa errada estava sendo selecionada, permitindo identificar o problema. Depois o operador de comparação foi corrigido e o teste foi executado novamente, confirmando o funcionamento correto do EDF, e ao longo do desenvolvimento, também foi realizado commits frequentes no Git, separando as principais funcionalidades implementadas e facilitando o acompanhamento das alterações realizadas no projeto.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

S1	Implementação incremental	O projeto foi desenvolvido por partes, implementando e testando cada funcionalidade antes de avançar para a próxima.	próxima. Não houve troca de estratégia.
----	---------------------------	--	--

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

1	s1	Implementação e teste da leitura das tarefas	ok		conforme evidencias.log 08/09/2026	evidencias.log
2	s1	Implementação da lista encadeada	ok		conforme evidencias.log 08/09/2026	evidencias.log
3	s1	Implementação do Rate Monotonic	ok		conforme evidencias.log 08/09/2026	evidencias.log

4	s1	Implementação do EDF	Falha → corrigido	corrigido Critério de comparação estava invertido, fazendo uma tarefa com deadline maior ser escolhida primeiro	conforme evidencias.log 08/09/2026 e print obrigatorio	Testes edf
5	s1	Verificação das tarefas ativas no fim da simulação	Falha → corrigido	O programa não contabilizava uma instância que ainda possuía tempo restante como KILLED	conforme evidencias.log 08/09/2026 e print obrigatorio	Teste KILLED
6	S1	Implementação dos períodos de ociosidade	OK		conforme evidencias.log 08/09/2026	TESTES IDLE
7	S1	Testes de perda de deadline e novas instâncias	OK		conforme evidencias.log 08/09/2026	testes de deadline
8	S1	Validação de entradas inválidas	OK		conforme evidencias.log 08/09/2026	TESTES DE VALIDAÇÃO

6. Evidências

Meu evidencias log esta la, da maneira certa com todos os testes.

6.1 Log automático (feito)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

PRINTS OBRIGATÓRIOS:

```

luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/lfass - 3/Escalonamento-de-tarefas-cr-ticas-de-voo$ make
gcc -Wall -Wextra -std=c11 src/scheduler.c -o scheduler
luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/lfass - 3/Escalonamento-de-tarefas-cr-ticas-de-voo$ ./scheduler edf teste.txt
tempo=0 tarefa=A restante=6 deadline=10
tempo=1 tarefa=A restante=5 deadline=10
tempo=2 tarefa=A restante=4 deadline=10
tempo=3 tarefa=A restante=3 deadline=10
tempo=4 tarefa=A restante=2 deadline=10
tempo=5 tarefa=A restante=1 deadline=10
tempo=6 tarefa=B restante=8 deadline=8
tempo=7 tarefa=B restante=7 deadline=8
tempo=10 tarefa=A restante=6 deadline=20
tempo=11 tarefa=A restante=5 deadline=20
tempo=12 tarefa=A restante=4 deadline=20
tempo=13 tarefa=A restante=3 deadline=20
tempo=14 tarefa=A restante=2 deadline=20
tempo=15 tarefa=B restante=8 deadline=23
tempo=16 tarefa=B restante=7 deadline=23
tempo=17 tarefa=B restante=6 deadline=23
tempo=18 tarefa=B restante=5 deadline=23
tempo=19 tarefa=B restante=4 deadline=23
tempo=20 tarefa=A restante=6 deadline=30
tempo=21 tarefa=A restante=5 deadline=30
tempo=22 tarefa=A restante=4 deadline=30
tempo=23 tarefa=A restante=3 deadline=30
tempo=24 tarefa=A restante=2 deadline=30
tempo=25 tarefa=A restante=1 deadline=30
luisf@aluisio:/mnt/c/Users/luisf/FACULDADE/lfass - 3/Escalonamento-de-tarefas-cr-ticas-de-voo$

```

Estratégia: comparar o resultado observado com o comportamento esperado para um caso simples e controlado, isolando a regra de prioridade do EDF para localizar e corrigir a falha lógica.

```

EXECUTION BY RATE
[A] for 6 units - F
[B] for 4 units - H
[A] for 6 units - F
[B] for 2 units - F
LOST DEADLINES
[A] 0
[B] 0
COMPLETE EXECUTION
[A] 2
[B] 1
KILLED
[A] 0
[B] 0

```

o programa não fazia nenhuma verificação das tarefas que ainda estavam ativas.

Então adicionamos, logo depois do while, uma verificação
A lógica é:

ativa == 1 → a tarefa ainda estava concorrendo/executando;
restante > 0 → ela não terminou;
a simulação acabou → então essa instância foi KILLED.

Depois rodamos novamente e obtivemos:

KILLED
[A] 1

7. Uso de IA

Usei IA para tirar dúvidas durante a implementação e entender alguns problemas de execução e lógica. Também usei para ajudar a definir testes para verificar os algoritmos Rate e EDF e comparar os resultados, e principalmente para me tirar do 0 do aprendizado.

Prompts principais:

- Como tratar a perda de deadlines na simulação?
- Como identificar tarefas que ainda estão ativas no final da simulação?
- Me mande vários testes para o meu projeto.
- Me ajude a montar o README e o relatório do projeto.

validei manualmente compilando o projeto com make e executando os algoritmos Rate e EDF com diferentes entradas. depois avaliei com testes gerados por ia para automatiza os testes demais, Também conferi os arquivos de saída para verificar se as tarefas eram executadas na ordem correta, se ocorria preempção, perda de deadlines, períodos de ociosidade e tarefas KILLED. Testei também empates entre tarefas, deadlines exatamente no momento da conclusão e diferentes tipos de entradas inválidas, como texto, zero, números negativos, $C > D$, $D > P$, campos faltando e arquivo inexistente.

8. Reflexão Final

Uma das principais decisões técnicas foi usar uma lista encadeada para armazenar as tarefas e manter Rate e EDF no mesmo programa. Também foi importante controlar as instâncias, os deadlines, as preempções e as tarefas que permaneciam ativas no final. Durante o desenvolvimento, percebi que os casos de perda de deadline precisavam de bastante atenção para não contabilizar a instância de forma errada. Na próxima vez, eu planejava desde o início mais casos específicos para testar cada regra do escalonador. Também evitaria deixar algumas correções para o final, principalmente nas partes relacionadas à contagem de KILLED e LOST DEADLINES.

9. Checklist Final de Entrega

Compilei do zero seguindo só o que está no relatório	ok
Rodei os testes e evidencias.log foi gerado	ok
Tenho prints obrigatórios	ok
Testei pelo menos um caso limite e um caso inválido	ok
Preenchi a seção de uso de IA	ok
Revisei o relatório e removi frases genéricas / vazias	ok

10. Se eu tivesse mais 2 horas

Se eu tivesse mais 2 horas, eu faria mais testes com entradas diferentes para tentar encontrar algum caso que ainda pudesse apresentar comportamento inesperado. Também revisaria o código para verificar se todas as regras do enunciado estão sendo atendidas corretamente. Daria uma atenção maior aos casos de preempção e perda de deadlines, principalmente quando várias tarefas chegam no mesmo instante. Compararia mais resultados entre Rate e EDF para encontrar outros exemplos além dos testes já realizados. Também revisaria a documentação e organizaria melhor as evidências dos testes realizados.